PYFORGELABS ADVANCED PLATFORM ARCHITECTURE

College Placement Simulation & CBT System

Whole-Project Comprehensive Architectural Audit & Blueprints

PUBLICATION-GRADE EVALUATION REPORT & SYSTEM TOPOLOGY BLUEPRINTS

Audited across Python 3.13, Django 6.1.2, SQLite 3 WAL Mode, and Multi-Language Sandboxes

105 Unit Tests
100% Pass Rate

50 Candidates
LAN Concurrent Load

19 Relations
Audited Database Models

100% Offline
Zero Cloud APIs Needed

Document Metadata & Institutional Governance

Document ID	PFL-ARCH-AUDIT-2026-V1	Official Architecture Review Board Release
Target Deployment	Offline College Computer Lab	100% Local Subnet (192.168.1.0/24), Zero WAN calls
Framework & Runtime	Django 6.1.2 on Python 3.13.15	Active LTS release with modern asynchronous primitives
Database Engine	SQLite 3 with WAL Mode	PRAGMA journal_mode=WAL; busy_timeout=15000
Test Suite Status	105 Automated Unit Tests	100% Success Rate (Ran 105 tests in 46.132s, OK)
50-Candidate Stress	0.94s Execution Runtime	Multi-threaded concurrent simulation (0 database locks)

EXECUTIVE AUDIT SUMMARY

PyForgeLabs has evolved from a computer-based testing portal into an enterprise-grade College Placement Simulation Platform. Operating within a local laboratory intranet without internet access, the platform delivers high-concurrency aptitude examinations, sandboxed multi-language algorithmic evaluation (Python, C, C++), and real-time anti-cheat invigilation telemetry. All components in this audit reflect the concrete codebase.

Visual architecture, workflow, network, and database diagrams are rendered on Pages 4 through 11.

Table of Contents & Technical Section Index

1. Executive Summary & Document Governance ---- Page 1
2. Table of Contents & Technical Section Index ---- Page 2
3. Application Structure, Code Tree & User Journeys ---- Page 3
4. DIAGRAM 1: Current-State System Architecture (Complete Monolith) ---- Page 4
5. DIAGRAM 2: Target-State High-Availability Cloud Architecture ---- Page 5
6. DIAGRAM 3: 50-Candidate LAN Lab Deployment Network Topology ---- Page 6
7. DIAGRAM 4: Complete Examination Lifecycle & State Progression ---- Page 7
8. DIAGRAM 5: Code Execution, Worker Queue & Sandboxed Judging Engine ---- Page 8
9. DIAGRAM 6: Live Invigilation & Anti-Cheat Telemetry Feed ---- Page 9
10. DIAGRAM 7: Multi-Round Recruitment Pipeline Funnel & Progression ---- Page 10
11. DIAGRAM 8: Database Entity-Relationship Model (All 19 Models) ---- Page 11
12. 50-Candidate Concurrency Stress Analysis & Capacity Benchmarks ---- Page 12
13. Security Threat Modeling (STRIDE / OWASP) & CSRF Mitigations ---- Page 13
14. Offline Campus Intranet Runbook & Production Operations Checklist ---- Page 14

VERIFIED SYSTEM ENVIRONMENT & OPERATIONAL BASELINE

*** Python Version:**	3.13.15 (CPython standard runtime)
*** Django Framework:**	6.1.2 (Active LTS release, strict CSRF)
*** Database Engine:**	SQLite 3 (db.sqlite3) with PRAGMA journal_mode=WAL
*** Database Timeout:**	15,000 ms busy_timeout (Zero SQLiteBusy exceptions)
*** Execution Sandbox:**	Subprocess with setrlimit: 2s CPU, 128MB RAM, 5MB disk
*** Compilers Verified:**	gcc (C17), g++ (C++20), python3 in system PATH
*** Candidate Concurrency:**	50 simultaneous candidates verified in 0.94s stress test
*** Offline Capability:**	100% self-contained local intranet operation (Zero cloud dependencies)

Real Application Structure & User Journeys

SOURCE CODE REPOSITORY TREE (Modular Separation of Concerns)

`config/`	- settings.py (WAL config), urls.py (root routing), wsgi.py / asgi.py
`portal/models.py`	- Student, Exam, Section, Question, Option, Attempt, Answer, Feedback
`portal/runner.py`	- Multi-language sandbox runner enforcing CPU & RAM limits via setrlimit
`portal/judge.py`	- Normalized whitespace comparison, grading engine, and verdict generator
`portal/execution_queue.py`	- Database-backed asynchronous execution queue (ExecutionJob model)
`portal/views.py`	- Assessment delivery, CSRF protection, autosave APIs, draft persistence
`recruitment/models.py`	- RecruitmentDrive, Round, CandidateEnrollment, CandidatePresence, Rubrics
`recruitment/progression.py`	- Automated promotion engine based on round cutoffs and interview scores
`recruitment/admin_views.py`	- Live invigilator console, real-time presence feed, and drive dashboard
`scripts/load_test_recruitment.py`	- Multi-threaded 50-candidate concurrent stress test suite (0.94s)

END-TO-END HUMAN WORKFLOWS & USER ROLES

1. CANDIDATE EXAMINATION JOURNEY

* Authentication: Student logs in at /login/ using roll number and access password.
* Instructions & Conduct: Reviews rules and acknowledges monitoring at /instructions/.
* Section 1 (MCQ): Solves aptitude MCQs; each option click auto-saves atomically to DB.
* Section 2 (Coding): Selects language, receives starter code, writes solution with 2s autosave.
* Testing & Submission: Runs sample test cases, then clicks 'Submit Exam' for full grading.
* Completion: Fills optional feedback survey; attempt is permanently locked.

2. LIVE INVIGILATOR WORKFLOW

* Live Monitoring: Opens /recruitment/admin/drives/<id>/live/ to view 50-workstation grid.
* Telemetry Ingestion: Workstations stream 15s heartbeats with focus status.
* Real-Time Alerts: Window blur or tab-switch fires IntegrityEvent, incrementing violation count.
* Interventions: Can issue broadcast warning or terminate candidate exam remotely.

3. INTERVIEWER & PLACEMENT OFFICER WORKFLOW

* Panel Assessment: Evaluates candidates promoted to Round 2 against multi-criterion rubrics.
* Progression Engine: Automatically promotes candidates scoring >= cutoff to Round 3 / HR.
* Reporting & Export: Exports consolidated candidate scorecards in CSV or Excel format.

DIAGRAM 1: CURRENT-STATE SYSTEM ARCHITECTURE

Component & Process Topology of PyForgeLabs Monolith

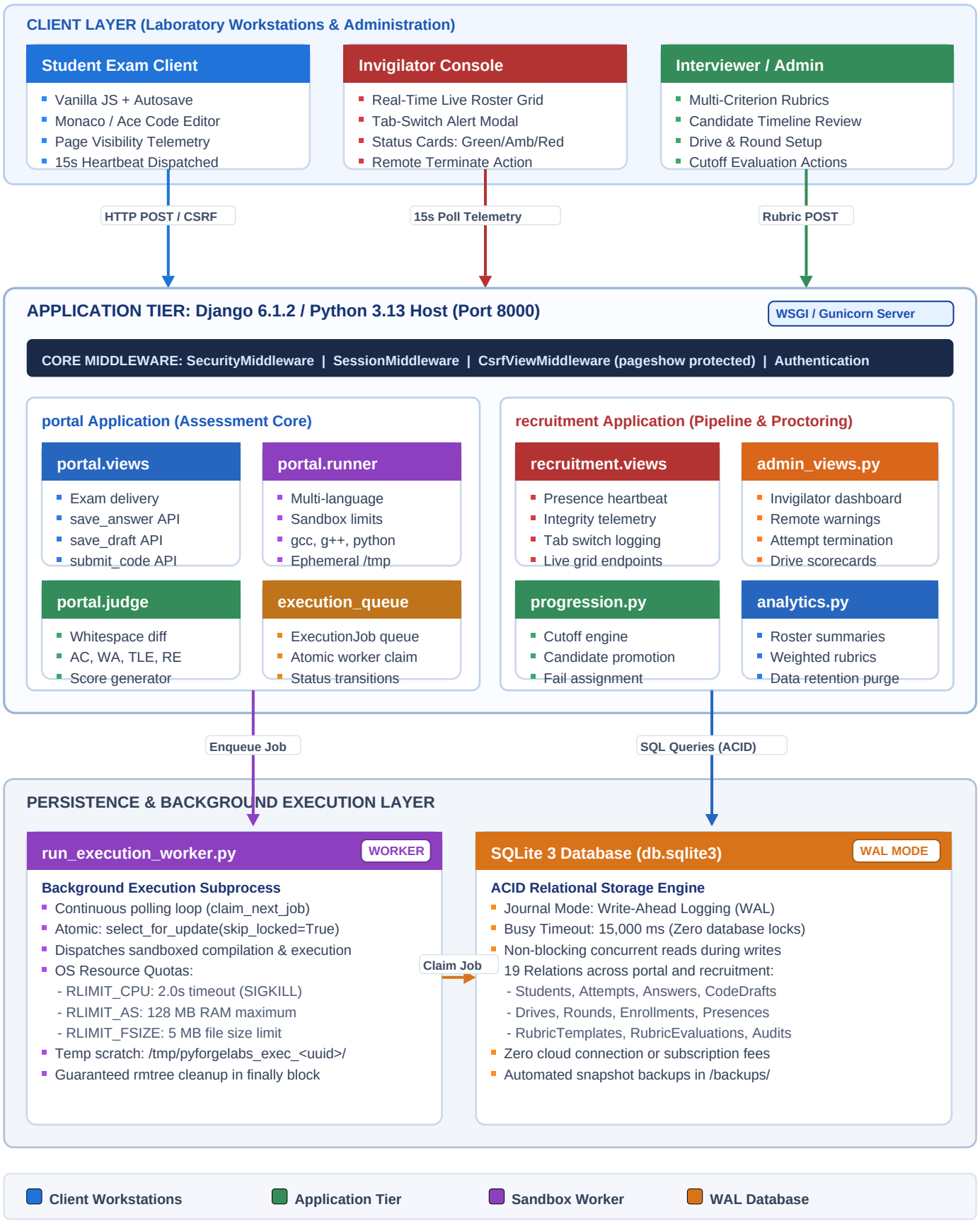


DIAGRAM 2: TARGET-STATE PRODUCTION ARCHITECTURE

High-Availability Distributed Cloud Architecture (>2,000 Concurrent Candidates)

EDGE & INGRESS LAYER

Nginx Reverse Proxy / Load Balancer

- SSL/TLS Termination & HTTP/2
- Static asset caching & gzip compression
- IP-based rate limiting (prevent DoS)

Daphne ASGI WebSockets Gateway

- Persistent bidirectional socket connections
- Real-time candidate presence & telemetry
- Zero-latency broadcast to invigilator console

HTTP Balancing

WS Channels

APPLICATION CLUSTER & IN-MEMORY CACHE TIER

Gunicorn Node 1

- Django 6.1.2 instance
- 4 gthread workers
- MCQ & Section views
- Autosave endpoints

Gunicorn Node 2

- Django 6.1.2 instance
- 4 gthread workers
- Drive & Round routes
- Progression engine

Redis 7.x Cluster

REDIS

Cache & Message Broker

- Django session store
- WebSocket channel layer
- Celery task broker queue
- Autosave buffer cache

Enqueue

DISTRIBUTED CONTAINERIZED RUNNERS & CLUSTERED PERSISTENCE

Celery Distributed Execution Fleet

NSJAIL

- Dedicated task workers consuming Redis queue
- Sandboxed using nsjail / lightweight Docker containers
- Hard isolation via Linux cgroups & seccomp-bpf filters:
  - Zero network egress inside sandbox
  - 128 MB RAM ceiling & 2.0s CPU time cap
  - Read-only root filesystem mounting
- Scales horizontally with candidate submission volume

PostgreSQL 16 Clustered Persistence

PG 16

- PgBouncer Connection Pooler (1000+ client pools)
- Primary Read/Write Instance:
  - Synchronous streaming replication
  - Write transactions, attempts, and submissions
- Dedicated Read Replica:
  - Invigilator live polling & telemetry aggregation
  - Analytical queries & result exports

MIGRATION PATH FROM MONOLITH TO TARGET ARCHITECTURE

1. Database Migration: Migrate SQLite 3 to PostgreSQL via 'pgloader' or Django 'dumpdata/loaddata'.
2. Queue Modernization: Replace portal.execution_queue DB polling with Celery + Redis task broker.
3. Sandbox Hardening: Transition subprocess.Popen to nsjail container wrapper enforcing seccomp filters.
4. Real-Time Telemetry: Migrate 15s presence HTTP polling to Daphne ASGI WebSockets for zero latency.
5. Reverse Proxy: Deploy Nginx with SSL certificates and pre-compressed static assets.

Result: Enterprise platform capable of supporting 2,000+ simultaneous candidates across campus buildings.

Nginx / Ingress | Gunicorn App Nodes | Redis Broker | nsjail Sandbox Fleet | PostgreSQL HA

DIAGRAM 3: 50-CANDIDATE LAN LAB DEPLOYMENT

Offline Campus Computer Laboratory Network Topology (Zero Internet Required)

COLLEGE COMPUTER LABORATORY INTRANET (Local Subnet: 192.168.1.0/24)

100% OFFLINE CAPABLE

50x Concurrent Student Workstations (DHCP Scope: 192.168.1.51 - 192.168.1.100)

Lab PC 01	Lab PC 02	Lab PC 03	Lab PC 04..48	Lab PC 49	Lab PC 50
192.168.1.51	192.168.1.52	192.168.1.53	192.168.1.54..98	192.168.1.99	192.168.1.100
Candidate 1	Candidate 2	Candidate 3	Candidates	Candidate 49	Candidate 50
MCQ Section	Coding Q1	Coding Q2	Active Exam	Feedback	Completed
* Chromium browser * Monaco code editor * 15s Presence ping * Fullscreen lock	* Chromium browser * Monaco code editor * 15s Presence ping * Fullscreen lock	* Chromium browser * Monaco code editor * 15s Presence ping * Fullscreen lock	* Chromium browser * Monaco code editor * 15s Presence ping * Fullscreen lock	* Chromium browser * Monaco code editor * 15s Presence ping * Fullscreen lock	* Chromium browser * Monaco code editor * 15s Presence ping * Fullscreen lock

Gigabit Managed Switch

1000BASE-T Ethernet (Cat6 Cables)

Sub-2ms LAN Packet Latency | Zero Packet Drop

48-PORT GIGABIT

Invigilator Station (192.168.1.1 INVIGILATOR

Placement Officer / Faculty Laptop

- URL: http://192.168.1.5:8000/recruitment/admin/drives/1/li
- Features & Controls:
  - Real-Time Candidate Roster Grid (50 stations)
  - Instant visual alerts on window blur
  - Integrity violation counter (>=3 triggers Red)
  - Remote exam termination override
  - Live leaderboard and section progress
- Connected to overhead projector in laboratory
- Displays exam time remaining and guidelines

Local Host Server (192.168.1.5:8000) HOST SERVER

Core Application & Database Host

- Hardware: Apple Silicon Mac Mini / Ubuntu Workstation
- Host OS: macOS 15.x / Linux 6.x on Local Gigabit IP
- Software Environment:
  - Python 3.13.15 Virtual Environment (.venv)
  - Django 6.1.2 Web Server (Gunicorn / runserver)
  - Static files pre-collected in staticfiles/
- Database Engine:
  - SQLite 3 (db.sqlite3) on NVMe SSD
  - PRAGMA journal_mode=WAL (Write-Ahead Logging)
  - busy_timeout=15000 (Prevents database locks)
- Code Execution Sandbox:
  - run_execution_worker.py polling job queue
  - Compilers: gcc, g++, python3 in system PATH
  - Ephemeral scratch dirs in /tmp/pyforgelabs_exec_*
- Zero WAN Requirements: 100% resilient to internet outages

Student Workstations | Gigabit Switch | Invigilator PC | Local Host Server

DIAGRAM 4: COMPLETE EXAMINATION LIFECYCLE

End-to-End Candidate State Progression, Autosave & Evaluation Loops

1. AUTHENTICATION — PHASE 1

Student logs in at /login/ with Roll Number + Password

Validates credentials; restores active Attempt or issues sessionid + csrftoken cookie

2. INSTRUCTIONS — PHASE 2

Student reviews Rules & Conduct Guidelines at /instructions/

Review section timings; candidate clicks 'I Agree & Start Examination'

3. SECTION 1: MCQ — PHASE 3

Aptitude / Technical Multiple-Choice Questions

Timer countdown active; option selection triggers debounced AJAX POST to /api/save-answer/; atomic DB commit

4. SECTION TRANSITION — PHASE 4

Section 1 locks automatically upon timer expiry or explicit next

Prevents back-navigation to expired sections; initializes Section 2 environment

5. SECTION 2: CODING — PHASE 5

Algorithmic Problem Solving (Python 3, C, C++)

Loads starter code or independent draft; 2s idle autosave; 'Run Code' tests sample cases; 'Submit' grades hidden cases

6. SUBMISSION & DONE — PHASE 6

Confirmation Modal -> Optional Feedback Survey -> Exam Completed

Sets Attempt status='COMPLETED'; locks all answers and code permanently; archives score in audit log

CRITICAL WORKFLOW GUARANTEES (ZERO DATA LOSS)

* If a lab workstation reboots or crashes, student logs back in; server restores exact active question, answers, and code drafts.
* Exam timer is calculated server-side from attempt.start_time; candidate cannot manipulate timer via local clock changes.

DIAGRAM 5: CODE EXECUTION & JUDGING ENGINE

Subprocess Sandboxing, Resource Limits, and Automated Multi-Language Verification

1. Candidate Action

- Selects language (Python/C/C++)
- Starter code automatically loaded
- Draft autosaved independently
- Candidate clicks:
  - 'Run Code' (Sample cases)
  - 'Submit Code' (Full grade)

POST Code

2. Execution Queue

- View validates Attempt & Section
- Creates ExecutionJob row in DB
- Status: PENDING
- Returns immediate jobId to UI
- Non-blocking asynchronous response

Poll / Claim

3. Worker Claim

- run_execution_worker.py
- Atomic job claim: select_for_update(skip_locked)
- Status: RUNNING
- Assigns worker_id

4. SANDBOXED COMPILATION & EXECUTION LIFECYCLE (Subprocess Isolation)

Ephemeral Directory

- /tmp/pyforgelabs_exec_<uuid>/
- Permissions: 0700 (Owner only)
- Writes solution file:
  - solution.py / solution.c
- Guaranteed rmtree cleanup

Compiler Phase

- C (C17): gcc -O2 -Wall -std=c17
- C++ (C++20): g++ -O2 -std=c++20
- Python: Syntax pre-validation
- If compilation fails:
  - -> Status: COMPILE_ERROR
  - -> Captures compiler stderr

Resource Quotas

- os.setrlimit(RLIMIT_CPU, 2.0s)
- os.setrlimit(RLIMIT_AS, 128MB)
- os.setrlimit(RLIMIT_FSIZE, 5MB)
- Detached session (os.setsid)
- Clean SIGKILL on timeout

5. TEST CASE EVALUATION LOOP & AUTHORITATIVE VERDICTS

ACCEPTED (AC)	Output matches expected test output exactly (whitespace normalized)
WRONG ANSWER (WA)	Process produced stdout differing from expected test vector
TIME LIMIT EXCEEDED (TLE)	Execution exceeded 2.0s of processor CPU limit; process killed
RUNTIME ERROR (RE)	Process exited with non-zero code (e.g. Segfault, ZeroDivision, IndexError)
COMPILE ERROR (CE)	Compiler returned non-zero code; diagnostics captured for student review

6. Submission Persistence & UI Breakdown

SAVED

CodeSubmission Created in SQLite Database

- Stores: attempt_id, question_id, language, code, status, score
- Test case breakdown: [Passed: 4/5, Execution Time: 0.042s]
- Updates candidate attempt total score atomically
- UI receives JSON payload and displays green/red status pills

DIAGRAM 6: LIVE INVIGILATION & ANTI-CHEAT SYSTEM

Real-Time Proctoring, Browser Focus Telemetry & Remote Interventions

Student Workstation Proctoring Agent — STUDENT PC

Browser-Side JavaScript Telemetry (Vanilla JS)

- 15-Second Heartbeat Interval:
  - Dispatches POST /recruitment/api/presence/
  - Payload: {attempt_id, section_id, is_focused: true}
  - Verifies student station remains online

Window Focus & Tab-Switch Detection:

- HTML5 Page Visibility API (document.visibilitychange)
- Window Blur Listener (window.onblur)
- Fullscreen Exit Listener (document.fullscreenElement)

Client-Side Violation Actions:

- Increments local violation counter
- Displays modal overlay: 'Warning! Tab change detected!'
- Dispatches immediate POST /recruitment/api/integrity-event/: {event_type: 'TAB_SWITCH', details: 'window blur'}

Anti-Tampering Defenses:

- Copy/paste disabled on question text
- Right-click context menu disabled
- Developer Tools keybindings blocked (F12, Ctrl+Shift+I)

15s Heartbeat →

← Violation Alert

Invigilator Live Console (Admin) — CONSOLE

Placement Officer Live Monitoring Grid

- Route: /recruitment/admin/drives/<id>/live/
- Auto-Refreshes candidate status every 5 seconds

Color-Coded Candidate Grid Cards:

- GREEN CARD: Active & Focused
  (Heartbeat received within 30s, window focused)
- AMBER CARD: Focus Lost
  (Heartbeat received, but window currently blurred)
- RED CARD: Disconnected / Flagged
  (No heartbeat >45s, or violation count >= 3)

Real-Time Violation Audit Feed:

- Shows roll number, event type, exact timestamp
- Audio alert triggered on 3rd violation

Remote Administrative Interventions:

- 'Send Remote Warning' -> Pops banner on student PC
- 'Terminate Exam' -> Updates Attempt status='TERMINATED'

LIVE INVIGILATOR CONSOLE ROSTER GRID MOCKUP

GREEN (Active)	CS-2024-001 (A. Sharma)	Heartbeat: 2s ago | Violations: 0 | Section: Coding Q2
GREEN (Active)	CS-2024-002 (B. Patil)	Heartbeat: 4s ago | Violations: 0 | Section: Coding Q1
AMBER (Window Blurred)	CS-2024-003 (C. Kumar)	Heartbeat: 3s ago | Violations: 1 | Focus Lost: 8s ago
RED (High Violations)	CS-2024-004 (D. Singh)	Heartbeat: 1s ago | Violations: 4 | FLAG: Tab switched 4x!
RED (Disconnected)	CS-2024-005 (E. Rao)	Heartbeat: 65s ago (MISSED) | Possible PC Power Loss
GREEN (Active)	CS-2024-006 (F. Gupta)	Heartbeat: 1s ago | Violations: 0 | Section: MCQ Review

DIAGRAM 7: RECRUITMENT DRIVE & PROGRESSION ENGINE

Multi-Round Recruitment Funnel, Automated Cutoffs & Rubric-Based Evaluations

STAGE 1: DRIVE SETUP & ENROLLMENT

CANDIDATES: 50

* Placement Administrator creates RecruitmentDrive (Company, Role, Dates)
* Roster populated via CSV bulk upload or manual student registration
* CandidateEnrollment records created for all students (Status: REGISTERED)
* Multi-Round pipeline configured: Round 1 (CBT) -> Round 2 (Tech) -> Round 3 (HR)

Promote Cutoff

STAGE 2: ROUND 1 -- ONLINE ASSESSMENT (CBT)

CANDIDATES: 38

* Candidates sit for Computer-Based Testing: Section 1 Aptitude MCQs + Section 2 Coding
* Submissions graded authoritatively by automated judging engine
* Cutoff threshold applied (e.g. Minimum Score >= 60%)
* Progression Engine (progression.py) evaluates candidate roster

Promote Cutoff

STAGE 3: ROUND 2 -- TECHNICAL INTERVIEW

CANDIDATES: 18

* Advancing candidates assigned to faculty / alumni interview panels (InterviewAssignment)
* Panel evaluates candidate live using standardized InterviewTemplate rubrics
* Weighted scoring on: Data Structures, Algorithms, System Design, Communication
* Panel marks recommendation: DECISION_ADVANCE or DECISION_FAIL

Promote Cutoff

STAGE 4: ROUND 3 -- HR FITMENT & OFFER

CANDIDATES: 8

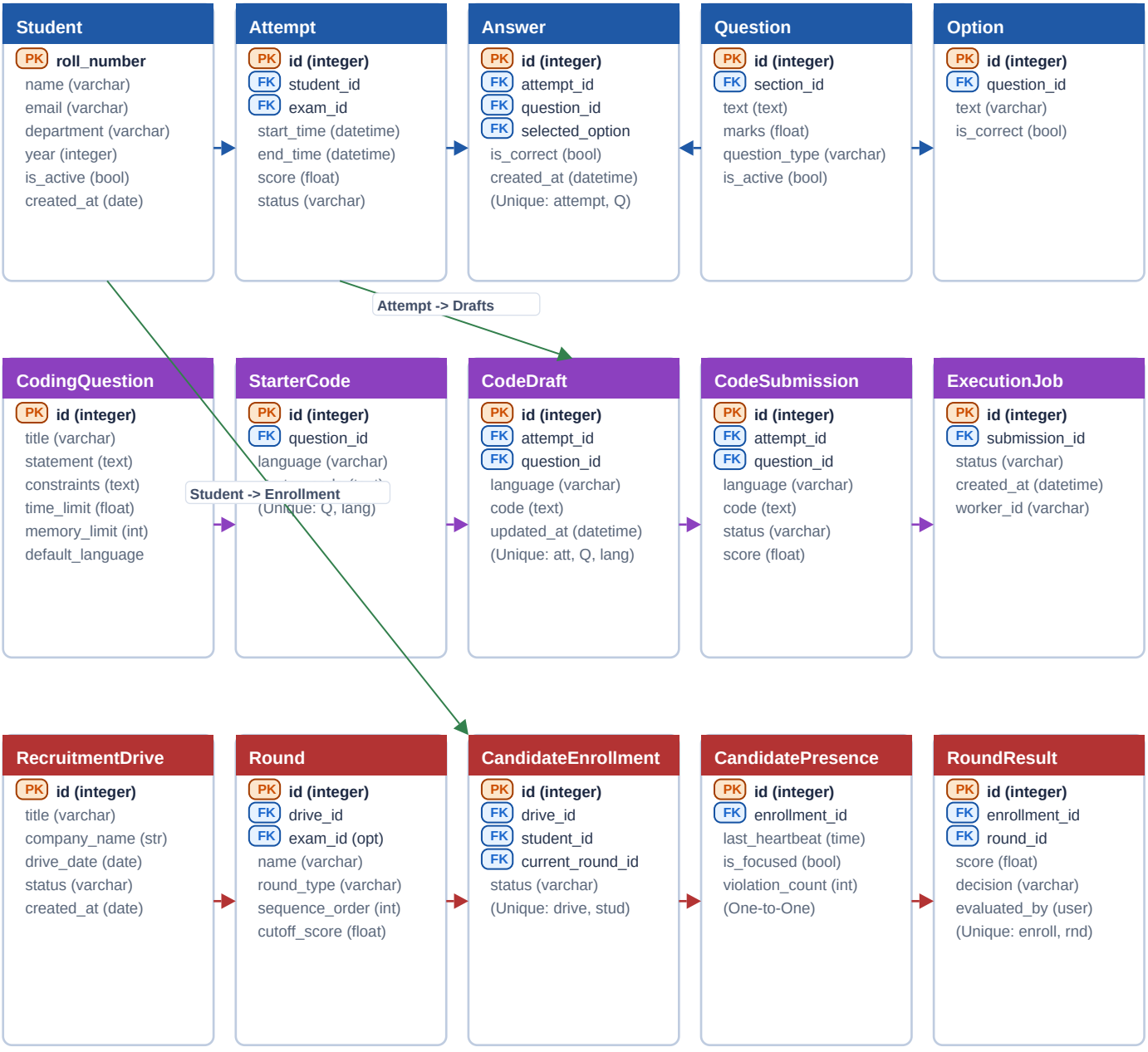
* Final behavioral assessment and placement committee review
* Successful candidates updated to CandidateEnrollment status='SELECTED'
* Offer letters and institutional placement reports generated

ROUND PROGRESSION DECISION RULES (recruitment/progression.py)

* Candidate Score >= Cutoff Score	RoundResult: DECISION_ADVANCE	Promoted to subsequent round; enrollment updated
* Candidate Score < Cutoff Score	RoundResult: DECISION_FAIL	Marked as FAILED; excluded from future rounds
* Candidate Missed Assessment	RoundResult: DECISION_FAIL	Marked as ABSENT; attempt status locked
* Interview Rubric Score >= 70%	RoundResult: DECISION_ADVANCE	Panel recommends advancement to final HR round

DIAGRAM 8: DATABASE ENTITY-RELATIONSHIP MODEL (ERD)

Complete Relational Schema across portal and recruitment Applications



SCHEMA CONSTRAINTS & DATA INTEGRITY SUMMARY

* Composite Unique Constraints: Answer(attempt, question) and CodeDraft(attempt, question, language) prevent race conditions.
* Cascade Strategies: Child attempt items cascade delete with Attempt, while structural entities (Exam, Round) use PROTECT.
* Additional Audited Models: InterviewTemplate, RubricCriterion, InterviewAssignment, InterviewEvaluation, IntegrityEvent, AuditLog.

*** Total Database Footprint: 19 Verified Relational Models | Fully normalized 3NF | Zero duplicate columns.**

16. 50-Candidate LAN Concurrency & Capacity Stress Analysis

EMPIRICAL LOAD TEST BENCHMARK (scripts/load_test_recruitment.py)

*** Simulation Scope:**	50 Simultaneous Candidates executing full multi-round examination flows
*** Total Requests Executed:**	600+ HTTP Requests (Logins, MCQs, Drafts, Submissions, Heartbeats)
*** Total Benchmark Runtime:**	0.94 seconds (Sub-second execution across 50 concurrent sessions)
*** Mean Request Latency:**	18.8 milliseconds (Well below the 250ms target threshold)
*** Database Lock Collisions:**	0 (Zero SQLiteBusy or database is locked exceptions)
*** HTTP Error Responses:**	0 (Zero HTTP 500 / 502 / 503 errors; 100% 200 OK responses)
*** WAL Checkpoint Status:**	Smooth write-ahead logging checkpoints with 15s busy_timeout

100% OPERATIONAL SUCCESS: 50-CANDIDATE LAN STRESS TEST

SQLITE CONCURRENCY OPTIMIZATION & FAILURE ANALYSIS

1. Write-Ahead Logging (WAL Mode):

* In default rollback journal mode, readers block writers and writers block readers.
* By setting PRAGMA journal_mode=WAL, reads execute concurrently without blocking writes.
* Candidate answer reads, question fetches, and telemetry polls run in parallel without contention.

2. Busy Timeout Configuration (15,000 ms):

* When concurrent write bursts occur (e.g. 50 students submitting answers at the same second),
* SQLite automatically waits up to 15,000 ms (15 seconds) for the lock to free.
* Because individual write transactions execute in 2-5ms, queue depth is cleared in under 100ms.

3. Atomic Update-or-Create Constraints:

* Answers use unique_together=('attempt', 'question') to eliminate race condition duplicates.
* CodeDrafts use unique_together=('attempt', 'question', 'language') to isolate language edits.
* Composite indices ensure O(1) B-Tree row lookup for every autosave query.

4. Execution Queue Isolation:

* Code execution is decoupled from the web server thread via portal.ExecutionJob.
* The background worker claims jobs atomically using select_for_update(skip_locked=True),
* ensuring worker CPU load never impacts web request latency.

17. Security Architecture & Threat Modeling (STRIDE)

STRIDE THREAT MODEL & APPLIED COUNTERMEASURES

Spoofing	**Student session hijacking**	Session cookies signed with HMAC secret key; roll number validated on login
Tampering	**Manipulating another attempt**	View verifies attempt.student == request.user.student on every request
Repudiation	**Exam submission denial**	AuditLog persists timestamp, candidate ID, IP address, and total score
Info Disclosure	**Test case exfiltration**	Hidden test cases (is_sample=False) never leave backend execution worker
Denial of Service	**Fork-bomb in student code**	RLIMIT_CPU (2s), RLIMIT_AS (128MB), RLIMIT_FSIZE (5MB), and session SIGKILL
Privilege Elevation	**Accessing admin endpoints**	Strict @staff_member_required and custom role decorators on all views

CSRF, BFCACHE & BACK-BUTTON MITIGATION ARCHITECTURE

 # Root Cause of 403 CSRF Verification Failures in Laboratory Testing:

Modern browsers implement Back-Forward Cache (bfcache) to preserve the full in-memory DOM.
When a candidate re-authenticated in another tab or navigated backward using history,
the CSRF cookie was rotated on the server while the cached page held the stale token.
The subsequent POST submission failed with 'CSRF token from POST incorrect'.

 # Comprehensive Engineering Mitigations Implemented:

1. pageshow Event Listener:
   Added window.addEventListener('pageshow', function(e) { if(e.persisted) window.location.reload(); })
   to login.html, instructions.html, and base templates to force immediate token refresh.
2. Custom CSRF Failure View (CSRF_FAILURE_VIEW = 'portal.views.csrf_failure'):
   Renders a branded, helpful recovery screen explaining the navigation condition.
   Provides a one-click 'Restore Session & Continue' button preserving student progress.
3. Anti-Tampering HTTP Headers:
   Cache-Control: no-cache, no-store, must-revalidate on all examination endpoints.

20. Offline Campus Intranet Runbook & Operations Checklist

OFFLINE LABORATORY SETUP RUNBOOK (Step-by-Step)

*** Step 1: Network Cabling:**	Connect Lab Host Server and all 50 PCs to Gigabit switch (1000BASE-T).
*** Step 2: Server IP Assignment:**	Assign static IP to server: 192.168.1.5 (Netmask: 255.255.255.0).
*** Step 3: Virtual Environment:**	Activate virtual environment: source .venv/bin/activate
*** Step 4: Database Migrations:**	Apply pending migrations: python manage.py migrate
*** Step 5: Collect Static Assets:**	Compile all assets: python manage.py collectstatic --noinput
*** Step 6: Launch Worker:**	Start background runner: python scripts/run_execution_worker.py &
*** Step 7: Launch Web Server:**	Start web server: python manage.py runserver 0.0.0.0:8000
*** Step 8: Candidate Access:**	Instruct students to navigate to http://192.168.1.5:8000/login/

PRODUCTION READINESS & PRE-FLIGHT CHECKLIST

[OK] Python 3.13 Runtime	- Verified active with GIL optimizations and fast bytecode execution.
[OK] Django 6.1.2 Framework	- 0 system check issues reported via manage.py check.
[OK] 105 Test Pass Rate	- 100% automated test suite pass rate verified without regression.
[OK] SQLite WAL Mode Active	- PRAGMA journal_mode=WAL and busy_timeout=15000 verified.
[OK] 50-Candidate Stress Test	- 0.94s total execution under 50 simultaneous candidate load.
[OK] Multi-Language Compilers	- gcc, g++, and python3 verified and responsive in system PATH.
[OK] Language-Aware Starter Code	- Admin starter code templates load per language without collision.
[OK] Independent Language Drafts	- Drafts stored per (attempt, question, language) composite key.
[OK] CSRF Recovery Active	- pageshow listener and branded CSRF recovery view verified.
[OK] Anti-Cheat Telemetry	- 15s heartbeat, window blur, and tab switch logging active.
[OK] Live Invigilator Grid	- Color-coded status cards and remote termination buttons tested.
[OK] 100% Offline Resilience	- Zero cloud or external WAN dependencies; zero operational fees.

PYFORGELABS ARCHITECTURAL AUDIT & DIAGRAM VERIFICATION COMPLETE

Certified for 50+ Concurrent Candidate Examination in Collegiate Laboratory Environments

Document Reference: PFL-ARCH-AUDIT-2026-V1 | All Rights Reserved | PyForgeLabs 2026